

Building C/C++ Applications

Dov Kruger

Department of Electrical and Computer Engineering
Rutgers University

February 9, 2026



Building C Programs with Multiple Files

- Real programs split across many `.c` / `.cpp` and headers
- Compiler turns each `.c`/`.cpp` into an object file (`.o` / `.obj`)
- Linker combines object files and libraries into one executable
- One definition per symbol; declarations can appear in many files



Functions and Function Prototypes

- **Definition:** function body (what it does)
- **Prototype:** name, return type, parameter types (no body)

add_prototype.c

```
// prototype (declaration) - often in .h  
int add(int a, int b);
```

```
// definition - in .c or .cpp  
int add(int a, int b) { return a + b; }
```



C/C++ Requires “Define First”

- Names must be **declared** before use (so compiler knows type)
- Definition can be in another file; declaration is enough to compile
- Forward declarations: declare a function (or type) before its definition
- Order matters within one file: use prototypes at top or include headers



Declare Before Use: OK

declare_ok.c

```
// OK: prototype before call  
int foo(int);  
int main() { return foo(0); }  
int foo(int x) { return x; }
```



Declare Before Use: Error

declare_bad.c

```
// Error: foo unknown at call site  
int main() { return foo(0); }  
int foo(int x) { return x; }
```



- **Global variable:** defined in one `.c/.cpp` (one definition)
- `extern` in others: “this symbol is defined elsewhere”
- Headers: use `extern` declarations; one `.c/.cpp` has the definition
- Without `extern` in a `.c` file you get a new definition → link errors



Global and extern Example

config.h

```
#ifndef CONFIG_H
#define CONFIG_H
extern int verbose;
#endif
```

config.c

```
#include "config.h"
int config = 42;
```

main_extern.c

```
#include "config.h"
int main(void) { if (verbose) return 1; return 0; }
```



- **Compilation:** `.c/.cpp` $\rightarrow$ `.o` (symbols may be undefined)
- **Linking:** `.o` + libraries $\rightarrow$ executable or library
- Linker resolves external symbols (functions, globals) to one definition
- Multiple definitions of same symbol $\rightarrow$ link error
- Unresolved symbol (used, never defined) $\rightarrow$ link error



Compiler Errors

- Syntax errors, type errors, undeclared names
- Reported per .c/.cpp at **compile** time
- Fix: add missing includes, prototypes, or fix types
- “Undefined reference” is usually a **link** error, not a compiler error

syntax_error.c

```
int main() {  
    int x = 5  
    // syntax error: missing semicolon  
    return 0;  
}
```

type_error.c

```
int main() {  
    int x = 5;  
    char* p = x; // type error: cannot convert int to char
```

- **Undefined reference:** symbol used but never defined in any .o or library
- **Multiple definition:** same symbol defined in more than one .o
- Fix: add the missing .c/.cpp or library to the link step; ensure globals are `extern` in headers and defined in one place
- First each file must be compiled, the link can only happen once all objects are known



Example errors

errors.c

```
#include <iostream>
using namespace std;

void bar();
void baz() { cout << "baz" << endl; }
void baz() { cout << "yo!" << endl; }
// compile error: multiple definition of 'baz'
int main() {
    int x = 3;
    int x = 4; // compile error: multiple definition of 'x'
    ,

    foo(); // compiler error, foo is not defined
    bar(); // linker undefined symbol error, bar is not
        defined
    // bar error would only be found once you fix the
        error with foo() above
    return 0;
}
```

C++ error examples

Find the errors in the following code:

errors.cpp

```
using namespace std;
int main() {
    cout << "Hello, World!" << endl;
    cout << x;
    int x = 3, y;
    cout << y;
    return 0
}
```

- Fix: define foo in a .cpp and link it, or link the library that contains foo
- Multiple definition: make one extern in header, define in one .cpp



- cout doesn't exist (because it is not included)
- x is not defined (because it is declared below the use)
- y is not initialized (which is not a compile error, but a runtime error)



Concept	Description
Rule format	<code>prog: prog.cpp</code> <code>g++ prog.cpp -o prog</code>
Default file	Makefile or makefile
<code>make</code>	Runs first rule in Makefile in current directory
<code>make -f myfile</code>	Use myfile instead
<code>make target</code>	Build named target
Multiple rules	<code>make</code> builds first target if none specified
Dependencies	<code>target: dep1 dep2</code>
Rebuild logic	Only if missing or older than dependency



Sample: Single-File Program and Makefile

prog.cpp

```
#include <iostream>
int main() {
    std::cout << "Hello\n";
    return 0;
}
```

Makefile

```
prog: prog.cpp
    g++ prog.cpp -o prog
```



More make Rules: prog and clean

Makefile

```
prog: prog.cpp
    g++ prog.cpp -o prog

clean:
    rm -f prog
```

- make builds prog (first target)
- make clean removes prog



Makefile

```
prog: prog.cpp
    g++ $< -o $@
```

- \$@ = target name (prog)
- \$< = first prerequisite (prog.cpp)
- Avoids repeating names in the recipe



make: Two Files Using Object Files

Makefile

```
prog: main_class.o foo.o
    g++ main_class.o foo.o -o prog

main_class.o: main_class.cpp foo.hpp
    g++ -c main_class.cpp -o main_class.o

foo.o: foo.cpp foo.hpp
    g++ -c foo.cpp -o foo.o

clean:
    rm -f prog main_class.o foo.o
```

- Compile each .cpp to .o; link .o files into prog
- -c: compile only, no link
- Rebuild only what changed (e.g. touch foo.cpp → only foo.o and prog rebuilt)



make: More Industrial-Strength

```
CXX := g++  
DEBUG := -g # put # before -g to disable debugging  
OPT := -O2 # put # before -O2 to disable optimization  
all: prog1 prog2  
  
%.o: %.cpp  
    $(CXX) $(DEBUG) $(OPT) -c $< -o $@  
  
prog1: a.o b.o  
    g++ $^ -o $@  
  
prog2: b.o c.o d.o  
    g++ $^ -o $@  
  
clean:  
    rm -f prog1 prog2 a.o b.o c.o d.o
```



Before inline functions: Macros

```
#include <iostream>

using namespace std;

#define SQUARE(x) x * x
#define max(a,b) ((a) > (b) ? (a) : (b))

int main() {
    int a = 2;
    cout << "a=" << a++ << endl;
    cout << "a=" << ++a << endl;
    int x = 5;
    cout << "SQUARE(x)=" << SQUARE(x) << endl;
    cout << "SQUARE(x++)=" << SQUARE(x++) << endl; //
        undefined behavior, side effects
    cout << "x=" << x << endl;
    int y = 3;
    y = max(y++, x++); // complete mess?
    cout << "x=" << x << ", y=" << y << endl;
    return 0;
}
```



C++: inline functions

- `inline`: hint to compiler to paste body at call site (no function call overhead)
- Efficient for small functions: no overhead of call/return
- Inline call where parameters are known can be much more efficient
- The compiler must know what the code is to inline it (must be defined already)

inline_example.cpp

```
inline uint64_t add(uint64_t a, uint64_t b) {  
    return a + b;  
}
```

```
uint64_t sub(uint64_t a, uint64_t b); // prototype,  
    implemented somewhere else
```

```
int main() {  
    uint64_t a = 1, b = 2;
```

When inline is a bad idea

- inline helps when the function is small
- If the function does a lot, the percentage of time in the call is tiny
- If the function is large, every call to inline would make executable bigger
- Rule of thumb: if the function is 1-3 lines, inline it

badinline.cpp

```
inline void doubleme(int x, std::vector<int> v) {  
    int n = v.size();  
    for (int i = 0; i < n; i++) {  
        v.push_back(i);  
    }  
    v.push_back(x); // stick x at the end  
}
```

```
int main() {  
    vector<int> v = {1, 2, 3, 4, 5};  
    doubleme(3, v); // not helpful
```

C++: Classes and Separate Compilation

- Class declaration in `.hpp` (or `.h`); members and method prototypes
- Method definitions in `.cpp`; one translation unit per `.cpp`
- Same model as C: compile each `.cpp` to `.o`, then link
- Include guard in header avoids multiple definition of classes



foo.hpp

```
#ifndef FOO_HPP
#define FOO_HPP
class Foo {
public:
    Foo(int x);
    int get() const;
    void set(int x);
private:
    int x_;
};
#endif
```



Why use #ifndef?

- If the header gets included twice, the same class is defined twice (error)
- to avoid this, test if the file has already been included and ignore the second time

a.hpp

```
#ifndef A_HPP
#define A_HPP
#include "C.hpp"
class A : public C { // details are irrelevant, just know
    our class depends on C
};
#endif
```

b.hpp

```
#ifndef B_HPP
#define B_HPP
```

C++ Class: Implementation and main

foo.cpp

```
#include "foo.hpp"
Foo::Foo(int x) : x_(x) {}
int Foo::get() const { return x_; }
void Foo::set(int x) { x_ = x; }
```

main_class.cpp

```
#include "foo.hpp"
int main() {
    Foo f(42);
    return f.get();
}
```



C++: Inline Functions

- `inline`: hint to compiler to paste body at call site (no function call overhead)
- Efficient for tiny functions: no call/return, often better code
- Definition usually in header so compiler can inline in every translation unit
- **Lack of code**: no separate `.cpp` needed; one place to maintain



Inline: One Header, No .cpp

inline_example.hpp

```
#ifndef INLINE_EXAMPLE_HPP
#define INLINE_EXAMPLE_HPP
// No separate .cpp: body in header, no call overhead
inline int sq(int x) { return x * x; }
#endif
```



Non-inline: Header + .cpp

non_inline.hpp

```
#ifndef NON_INLINE_HPP
#define NON_INLINE_HPP
// Declaration only; definition in .cpp
int sq(int x);
#endif
```

non_inline.cpp

```
#include "non_inline.hpp"
int sq(int x) { return x * x; }
```

